



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

JScamd: An Automated Static Taint Analysis Framework for Detecting Cryptographic API Misuses in JavaScript

Shijie Jia, State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences; School of Cyber Security, University of Chinese Academy of Sciences; Bowen Xu, Faculty of Electrical Engineering and Computer Science, Ningbo University; Yuan Ma, Yingjiao Niu, Limin Liu, and Daren Zha, State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences; School of Cyber Security, University of Chinese Academy of Sciences; Jingqiang Lin, School of Cyber Security, University of Science and Technology of China

<https://www.usenix.org/conference/usenixsecurity26/presentation/jia-shijie>

**This paper is included in the Proceedings of the
35th USENIX Security Symposium.**

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the
35th USENIX Security Symposium
is sponsored by



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

JScamd: An Automated Static Taint Analysis Framework for Detecting Cryptographic API Misuses in JavaScript

Shijie Jia^{†¶}, Bowen Xu^{‡✉}, Yuan Ma^{†¶}, Yingjiao Niu^{†¶}, Limin Liu^{†¶}, Daren Zha^{†¶}, Jingqiang Lin[§]

[†]State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, China

[‡]Faculty of Electrical Engineering and Computer Science, Ningbo University, China

[¶]School of Cyber Security, University of Chinese Academy of Sciences, China

[§]School of Cyber Security, University of Science and Technology of China, China

[†]{jia shijie, mayuan, niu yingjiao, liu limin, zha daren@iie.ac.cn}, [‡]{xubowen@nbu.edu.cn}, [§]{linjq@ustc.edu.cn}

Abstract

Cryptographic APIs are essential to application security, yet frequent developer misuse leads to widespread vulnerabilities. Detecting cryptographic misuse in JavaScript is challenging due to its dynamic semantics and fragmented cryptographic library ecosystem, which hinder static analysis tools from accurately modeling data flows and covering non-standard or evolving libraries, leading to high false-negative rates.

In this paper, we propose *JScamd*, a static analysis framework for scalable detection of cryptographic misuse in JavaScript. *JScamd* integrates an enhanced program representation engine, an LLM-assisted taint rule generation pipeline, and a precise static taint analysis engine to address JavaScript dynamic features. Building upon 16 systematically defined cryptographic security constraints, *JScamd* automatically analyzes cryptographic APIs, identifies security-sensitive taint sources and sinks, and synthesizes comprehensive taint analysis rules. This design enables accurate detection of cryptographic misuse across diverse JavaScript libraries and application scenarios. We performed a comprehensive large-scale evaluation on 322 widely adopted JavaScript projects. *JScamd* detects 1,059 misuses and achieves 97.78% precision, significantly outperforming three state-of-the-art tools. Our analysis further uncovers previously unknown vulnerabilities, resulting in 41 assigned CVEs. These results demonstrate that *JScamd* effectively bridges cryptographic security specifications and ecosystem-scale vulnerability detection in JavaScript.

1 Introduction

Cryptography is the foundation of modern information security. In practice, its capabilities are delivered to developers through cryptographic libraries (e.g., Java Cryptography Architecture and OpenSSL) and their application programming interfaces (APIs), which encapsulate complex cryptographic primitives into reusable components. Well-designed APIs allow developers to perform cryptographic operations,

such as encryption, decryption, and key management, without dealing with low-level implementation details. However, securely using cryptographic APIs remains challenging. Developers frequently struggle to configure security parameters correctly and comply with best practices, making cryptographic APIs highly susceptible to misuse [19, 26, 39]. JavaScript is a prominent example of this phenomenon. As one of the most influential programming ecosystems, JavaScript powers browser applications, Node.js-based server infrastructures, and numerous security-critical systems [2, 15, 16, 46]. Consequently, JavaScript projects increasingly rely on cryptographic libraries to protect data transmission, authentication, and user privacy across the web.

However, misuse of cryptographic APIs remains widespread, mainly due to the intrinsic complexity of cryptographic interfaces and developers relying on unreliable external guidance, such as misleading online discussions (e.g., Stack Overflow) and insecure code examples generated by AI tools [53]. Existing research has attempted to mitigate cryptographic API misuse through different technical approaches. Numerous static detectors such as CryptoGuard [39] and Gopher [53] have been proposed. However, research focusing on JavaScript remains scarce. First, JavaScript cryptographic ecosystem is highly fragmented, involving browser-native APIs [20], Node.js modules [34], and numerous third-party libraries [31, 43], which complicates automatic identification of cryptographic operations and construction of detection rules. Second, JavaScript dynamic features such as dynamic access, option object pattern and dynamic import, introduce fundamental challenges for misuse detection [18, 25, 37]. Third, existing tools such as Semgrep [22] and Njsscan [1] are widely used in practice, but they perform generic interprocedural data-flow analysis without explicitly modeling JavaScript-specific behaviors. Meanwhile, CodeQL [17] provides limited built-in support for cryptographic detection rules. As a result, these tools exhibit insufficient coverage when applied to real-world JavaScript projects.

To address these challenges, we present *JScamd*, an automated JavaScript cryptographic API misuse detection frame-

✉ Bowen Xu is the corresponding author.

work. Unlike existing tools that struggle with the dynamic features of JavaScript and diverse landscape of cryptographic libraries, *JScamd* integrates LLM-assisted rule generation with a specialized static taint analysis engine. This hybrid approach allows us to systematically formalize security constraints and adaptively identify misuses across diverse libraries. We validated *JScamd* on 322 real-world projects, demonstrating superior performance over state-of-the-art tools and identifying critical vulnerabilities.

In summary, this paper makes the following contributions:

- **Systematic Ecosystem Exploration.** We conduct a comprehensive study of JavaScript cryptographic libraries across browser and Node.js environments and formalize 16 JavaScript-specific cryptographic security constraints that cover prevalent high-impact misuse patterns.
- **Novel Automated Framework.** We develop *JScamd*, an automated static detection framework that combines LLM-assisted rule generation together with a JavaScript-aware taint analysis engine, enabling precise and scalable detection of cryptographic API misuses despite JavaScript dynamic semantics.
- **Large-Scale Empirical Analysis.** We evaluate *JScamd* on 322 popular projects, where it identifies 1,059 unique misuse cases with 97.78% precision—significantly outperforming Semgrep [22], Njsscan [1], and CodeQL [17]. Furthermore, our responsible disclosure process has resulted in 41 assigned CVE IDs, directly improving the security of the open-source ecosystem.

2 Background and Motivation

In this section, we provide the background of typical JavaScript features and present motivating examples to illustrate how these features pose unique challenges for cryptographic API misuse detection.

2.1 JavaScript Language Challenges

As a dynamic, prototype-based language, JavaScript offers developers high flexibility through runtime object manipulation, asynchronous programming, and weak typing [18, 25, 37]. However, these features significantly increase the complexity of static program analysis, especially in the context of detecting cryptographic API misuse.

Function Aliasing and Dynamic Access. JavaScript permits resolving function call targets via computed property access and variable assignment rather than static identifiers. In practice, cryptographic API invocations are often obscured by *function aliasing*, where methods are assigned to intermediate variables before invocation. For example, developers may assign the encryption function to a temporary variable (e.g., `const cipher = crypto.createCipheriv`) and invoke it

```

1 prepareEncryptKey() {
2   var lastKey = this.keys[this.keys.length - 1];
3   try {
4     if (this.prepared_key_original !== lastKey)
5     {
6       this.prepared_key_salt = CryptoJS.lib.
7         WordArray.random(256);
8       this.prepared_key_original = lastKey;
9       this.prepared_key = CryptoJS.PBKDF2(
10        lastKey.key, this.prepared_key_salt, {
11        hasher: CryptoJS.algo.SHA512,
12        keySize: 64 / 8,
13        iterations: 9,
14      });
15    }
16  } catch (e) {
17    console.log(e);
18  }
19  return this.prepared_key;
20 }

```

Figure 1: An insecure PBKDF key generation code snippet in the Twake project. The function invocation at Line 7 adopts the options object pattern.

indirectly (`cipher(alg, key, iv)`). Such indirection decouples the function definition from its call site, making it difficult for conventional static analyzers to reconstruct the call graph and reliably identify the underlying API. Consequently, pattern-matching tools (e.g., Semgrep) frequently fail to recognize these aliased calls, leading to false negatives.

Options Object Parameter Pattern. Unlike statically typed languages, JavaScript function definitions do not enforce parameter types, nor do they strictly validate the number of arguments at the call site. It is syntactically valid to invoke a function with more arguments than its formal definition specifies. Moreover, JavaScript facilitates the options object pattern to simulate named parameters. In this paradigm, developers define a function to accept a single configuration object rather than a list of distinct parameters (e.g., `createCipheriv(options)`, where the `options` object encapsulates the algorithm, key, and IV). At the call site, the caller passes an object literal where the key-value pairs serve as the actual arguments. While this approach improves code readability by grouping related configurations, it obscures the data flow for static analysis, as the function signature no longer explicitly reveals the required parameters (e.g., cryptographic keys or algorithm identifiers).

Dynamic Import And Module System. JavaScript supports dynamic module loading, where the arguments to `require()` or `import()` are computed at runtime rather than being static string literals. A common real-world pattern involves selecting a cryptographic provider based on external configurations, such as environment variables or user input. Since the specific module identifier (e.g., `const lib = require(config.useOpenSSL ?`

Table 1: Cryptographic constraints generated based on JavaScript multi-libraries. The Libs column indicates the libraries containing the corresponding cryptographic operations, where N denotes Node.js Crypto; W denotes the Web Cryptography API; C denotes CryptoJS; S denotes SJCL.

ID	Description	Libs	Classification
C-01	Do not use insecure cryptographic algorithms	N, W, C, S	Cryptographic
C-02	Do not use insecure Diffie-Hellman groups (modp1, modp2, modp5)	N	Algorithm Security
C-03	Do not use ECB mode for encryption	N, C	Cipher Mode Security
C-04	Do not use the non-random IV for CBC and CFB modes	N, W, C, S	
C-05	IV should be unique for block cipher algorithm modes	N, W, C, S	
C-06	Do not use insecure padding mode PKCS1-v1.5	N, W	
C-07	Do not use predictable/constant cryptographic keys	N, W, C, S	Key Lifecycle Security
C-08	Do not use predictable/constant salt for key derivation	N, W, C, S	
C-09	Do not use fewer than 1,000 iterations for key derivation	N, W, C, S	
C-10	Do not use fewer than 112 bits kLen for key derivation	N, W, C, S	
C-11	Do not use insecure PRNGs for cryptographic operations	Math.random	Random Number Security
C-12	Do not use insecure HTTP parser to tolerate malicious input	Node.js-http	Cryptographic Protocol Security
C-13	Do not establish a connection to the server with SSL/TLS invalid certificate	Node.js-tls	
C-14	Do not use weak SSL/TLS protocol versions	Node.js-tls	
C-15	JWT should be stored in an HttpOnly cookie	Cookie APIs	Secure Storage of
C-16	Session cookie should be subject to security controls by HttpOnly	Cookie APIs	Sensitive Parameters

"crypto" : "crypto-js")) is opaque to static analysis, standard tools cannot deterministically resolve the dependency graph. This ambiguity prevents the analyzer from associating the imported object with the correct API specifications, bypassing library-specific detections.

2.2 Motivating Example

The typical features of JavaScript and the variations in API designs across different cryptographic libraries present a range of technical challenges for detecting cryptographic API misuse. We provide a real-world code example to illustrate the motivation behind our work.

As shown in Figure 1, Line 7 invokes `CryptoJS.PBKDF2` with an insecure iteration count in Line 10. Despite the explicit vulnerability, state-of-the-art tools fail to detect it due to two fundamental limitations:

Limited Rule Coverage. Detectors such as Semgrep and CodeQL rely on manually curated rules to identify security-relevant sinks. While they can effectively detect generic misuses in well-known standard libraries (e.g., DES algorithm usage in the Node.js crypto module), they provide limited coverage for the large and diverse ecosystem of third-party libraries such as Stanford Javascript Crypto Library (SJCL). This limitation stems from the absence of unified or stable API specifications across third-party packages, as well as frequent variations in naming conventions, abstraction layers, and usage patterns. Moreover, such scanners often miss algorithm-specific configuration issues, for example, an insuf-

ficient iteration count in PBKDF2 (Line 7). As a consequence, achieving broad and precise coverage would require continuously creating and updating library-specific rules across thousands of evolving npm packages, a process that is inherently labor-intensive.

Inability to Handle Dynamic Feature. The flexibility inherent in JavaScript poses significant challenges for static vulnerability detection. As illustrated in Figure 1, Line 7 invokes the `PBKDF2` function from the `CryptoJS` library for key derivation. Notably, this invocation adopts the options object pattern for its third argument, where security-relevant parameters, including the hashing algorithm, derived key length, and iteration count, are encapsulated as key-value pairs within a configuration object. Existing static analysis tools, such as CodeQL, often fail to accurately model the data-flow relationships between nested object properties (e.g., the `iterations` field) and the callee `PBKDF2` function. As a result, constraints on the iteration count cannot be reliably enforced, leading to false negatives in misuse detection.

3 JavaScript Cryptographic Libraries and Misuse Constraints

In this section, we analyze JavaScript cryptographic libraries in terms of their implementation patterns, security properties, and cryptographic functionality, and derive a comprehensive set of misuse constraints for the JavaScript ecosystem.

Table 2: Detailed classification of cryptographic algorithm security constraint. The Libs column indicates the libraries containing the corresponding cryptographic operations, where N denotes Node.js Crypto; W denotes the Web Cryptography API; C denotes CryptoJS; S denotes SJCL.

ID	Description	Libs	Typical Weak Cryptographic Algorithms
C-01-a	Do not use insecure symmetric-key algorithms	N, C	DES, 2TDEA, 3TDEA, RC4
C-01-b	Do not use insecure asymmetric-key algorithms	N, W	RSA-512, RSA-1024, DSA-1024
C-01-c	Do not use insecure ECC algorithms	N, W, S	Field length ≤ 224 bits (e.g., secp192k1)
C-01-d	Do not use insecure hash algorithms	N, W, C, S	MD5, SHA-1, RIPEMD-160
C-01-e	Do not use potentially insecure HMAC algorithms	N, C	HMAC-MD5

3.1 JavaScript Cryptographic Libraries

JavaScript cryptographic libraries provide standardized interfaces for fundamental security operations, enabling developers to efficiently incorporate cryptographic functionality into software systems. To understand the cryptographic ecosystem in the JavaScript environment, we categorize commonly used cryptographic libraries in different scenarios. These libraries are classified based on their maintainers and intended application scope.

Official Standard Libraries. The JavaScript ecosystem provides two primary standard cryptographic libraries: the Web Cryptography API and the Node.js Crypto module. The Web Cryptography API defines a unified, strictly asynchronous interface standardized by the W3C for web browsers. The Node.js Crypto module, bundled with the Node.js runtime, provides JavaScript bindings to OpenSSL, exposing a comprehensive suite of cryptographic primitives. It is predominantly utilized in server-side environments and is characterized by its support for a vast array of algorithms and flexible parameter configuration.

Third-Party Libraries. In addition to official standards, the npm ecosystem hosts a large number of third-party cryptographic toolkits. Among these libraries, CryptoJS and SJCL (Stanford JavaScript Crypto Library) are widely used and serve as representative examples. These libraries are implemented entirely in native JavaScript and are compatible with both browser and Node.js environments. While they are often favored for their portability and ease of integration, they typically offer weaker security guarantees, such as permissive defaults or insufficient parameter validation, which increases the risk of cryptographic API misuse.

Several libraries are designed for lightweight or domain-specific use cases. For example, Brorand focuses on random number generation utilities, Node-RSA provides RSA-based encryption and decryption for Node.js applications, and libauth and ethereum-cryptography target blockchain systems such as Bitcoin and Ethereum. Although these libraries are effective within their intended domains, they often provide limited algorithm coverage and weaker security assurances, particularly when reused beyond their original application contexts.

3.2 Cryptographic Constraints

Building upon a systematic analysis of the usability and security of JavaScript cryptographic interfaces, we establish 16 misuse detection constraints that span six dimensions: cryptographic algorithm security, cipher mode security, key lifecycle security, random number security, cryptographic protocol security and secure storage of sensitive parameters, as shown in Table 1. Among these constraints, C-01 covers insecure cryptographic algorithms and requires a finer-grained decomposition because different algorithm families expose different security parameters and misuse patterns. Therefore, we further divide C-01 into five sub-constraints, covering insecure symmetric-key algorithms, asymmetric-key algorithms, elliptic-curve parameters, hash functions, and HMAC constructions, as summarized in Table 2. This decomposition allows JScamd to map high-level algorithm-security requirements to concrete JavaScript API patterns and parameter checks across different cryptographic libraries. These constraints are formulated with reference to up-to-date international standards and widely recognized security best practices [7, 12, 13, 20, 23, 24, 28, 44]. Furthermore, to account for the unique characteristics of JavaScript-based web development, we introduce additional constraints (e.g., C-15 and C-16) targeting cookie-related APIs, enhancing the security of sensitive parameter storage in web scenarios. Based on our systematic analysis of JavaScript cryptographic library APIs, each constraint is explicitly mapped to the corresponding API design and usage patterns, ensuring their practicality and applicability in real-world systems. The detailed explanations of the security rationale behind each constraint are provided in Appendix A.1.

4 Design of JScamd

In this section, we present the design of *JScamd*, starting with an overview and followed by detailed designs.

4.1 Overview

The *JScamd* framework is designed to bridge the gap between natural-language security constraints and static taint analysis

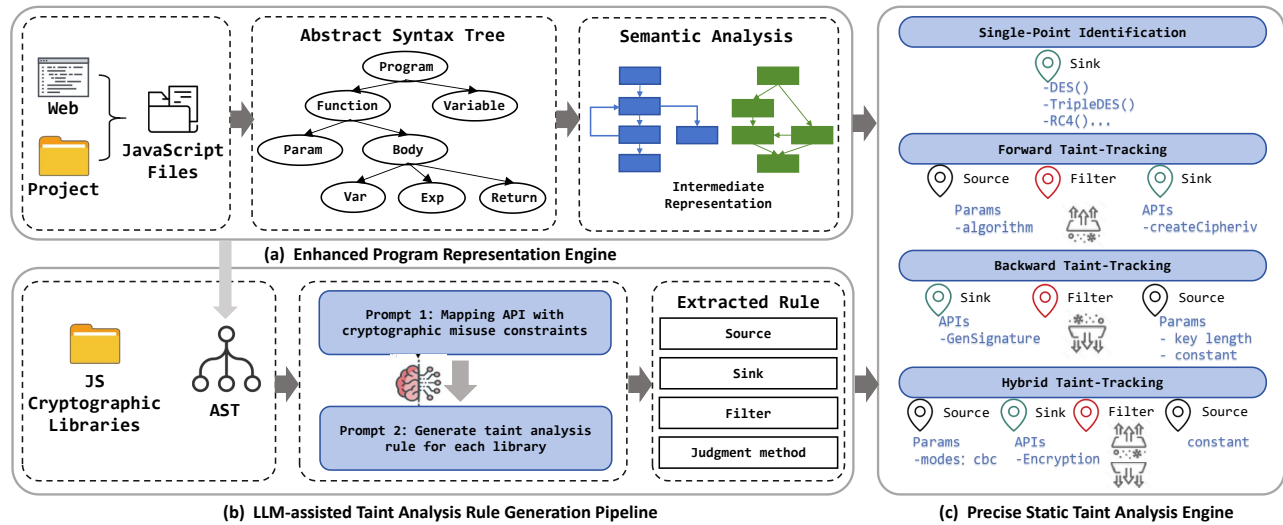


Figure 2: The framework of *JScamd*.

in the JavaScript ecosystem. Specifically, it aims to automatically detect cryptographic API misuse by linking textual security constraints with program-level data flow representations. As shown in Figure 2, *JScamd* is composed of three core components: (a) Enhanced program representation engine, (b) LLM-assisted taint analysis rule generation pipeline, (c) and Precise static taint analysis engine. Each component is designed to address the unique challenges of JavaScript dynamic features and heterogeneous cryptographic API landscape, enabling precise and automated misuse detection.

Enhanced Program Representation Engine. To enable precise static taint analysis for the detection of cryptographic API misuse in JavaScript, *JScamd* incorporates an enhanced program representation engine that transforms JavaScript source code into a lightweight cryptography-aware intermediate representation (IR). Starting from an abstract syntax tree (AST), the engine extracts semantic facts that are consumed by the taint analysis engine. Specifically, the engine tracks assignments and variable flows, normalizes JavaScript member expressions into canonical API paths, summarizes function parameters and return flows, and materializes call sites and object fields with lexical scope information. These IR facts explicitly capture security-relevant JavaScript semantics. By preserving scope-sensitive and field-sensitive data-flow information, the enhanced representation provides a precise foundation for downstream taint analysis.

LLM-assisted Taint Analysis Rule Generation Pipeline. To overcome the scalability and coverage limitations of manually written taint-analysis rules, *JScamd* incorporates an LLM-assisted rule generation pipeline that automatically transforms natural-language cryptographic API misuse descriptions into executable static taint rules. The pipeline combines cryptographic library documentation, structured misuse specifications, and static-analysis constraints to guide the model in

producing rules that conform to *JScamd* internal schema. By extracting API candidates, semantically mapping APIs to misuse definitions, and instantiating formal source–sink rules, *JScamd* enables rapid adaptation to new cryptographic libraries and evolving misuse patterns. To ensure correctness and executability, the generated rules are validated through automated consistency checks and an iterative self-correction loop, yielding a continuously extensible and up-to-date rule base for static cryptographic API misuse detection in the JavaScript ecosystem.

Precise Static Taint Analysis Engine. To accurately detect cryptographic API misuses in JavaScript, *JScamd* integrates a precise static taint analysis engine that tightly couples rule-driven reasoning with fine-grained propagation tracking. Guided by formally defined taint rules, the engine dynamically constructs inter- and intra-procedural propagation paths over the enhanced program representation, recording insecure data flows through blacklist and context-aware propagation structures. To accommodate diverse cryptographic usage patterns, *JScamd* supports multiple complementary taint analysis frameworks, including single-point identification, forward tracking, backward tracking, and hybrid analysis, each tailored to different API designs and misuse scenarios. By flexibly combining these analysis strategies, *JScamd* can reason about complex, multi-stage cryptographic behaviors and reliably identify security violations that are typically missed by conventional static analyzers in the presence of JavaScript dynamic features.

4.2 Enhanced Program Representation Engine

Taint analysis detects security violations by tracking flows from *sources* to *sinks* [38]. In the context of cryptographic misuse detection, *sources* typically correspond to insecure param-

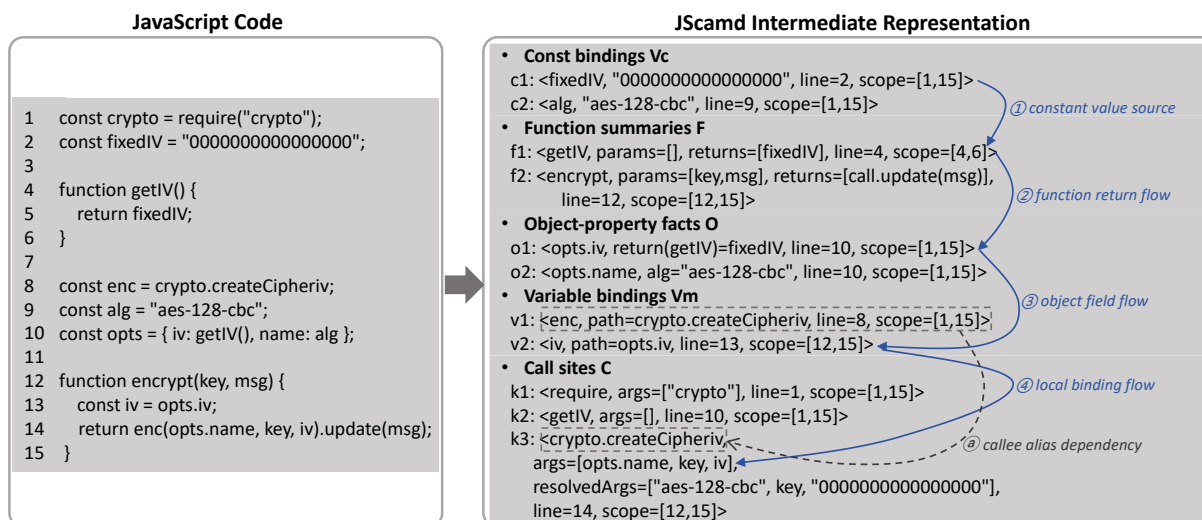


Figure 3: An example of JScamd IR-level dependency recovery. The right side shows a partial IR containing only the records relevant to this misuse, with their scope intervals from S . JScamd resolves the callee alias and links a hard-coded IV through a function return, an object field, and a local binding before it reaches the third argument of the normalized `createCipheriv` call.

eters (e.g., keys and IVs), while *sinks* denote cryptographic API invocations such as `crypto.createCipheriv()` or `subtle.encrypt()`. A misuse occurs when tainted data reaches a sink without satisfying required security constraints.

However, JavaScript dynamic features hinder conventional static analyzers from constructing precise dependency graphs. To address this challenge, *JScamd* introduces an enhanced program representation engine that compiles JavaScript programs into a lightweight cryptography-aware intermediate representation (IR). *JScamd* first uses Esprima [21], which supports ECMAScript syntax and preserves source-level structural information, to parse JavaScript source code into raw AST. However, raw AST are not directly suitable for security-oriented taint analysis because they lack the explicit data-flow and control-flow information required for detection. Therefore, *JScamd* transforms the raw AST into structured semantic facts that can be consumed by the taint analysis engine. This analysis-oriented representation preserves the security-relevant program facts required by cryptographic misuse rules.

(1) Design of Cryptography-Aware Intermediate Representation. *JScamd* accepts JavaScript code from individual files or project directories and parses them using Esprima. Leveraging these raw ASTs, *JScamd* then constructs an intermediate representation, where each analyzed program is represented as a collection of scope-annotated semantic records. An illustrative example of mapping source code to this IR is provided in Figure 3.

For a project P , the IR is organized as:

$$IR(P) = \langle V_c, V_m, F, C, O, S \rangle,$$

Each component captures one category of program informa-

tion required by the taint rules. The constant-binding component V_c records variables whose values can be statically resolved to literals, together with their source locations and valid lexical scopes. The variable-binding component V_m records assignments, aliases, and symbolic expressions whose values may depend on other variables, member paths, function returns, or composed expressions. The function-summary component F records each function’s name, formal parameters, return expressions, source location, and scope. The call-site component C records normalized callees, ordered arguments, object-literal arguments, source locations, and scopes. The object-field component O records property-level bindings extracted from object literals and member assignments. Finally, the scope component S records lexical intervals used to decide whether a definition is visible at a use site.

(2) Intermediate Representation Generation from Raw AST. *JScamd* processes the raw AST to generate IR for assignments, declarations, member expressions, object expressions, function definitions, and call expressions.

Step 1: Tracking Assignments and Variable Flows. The first step extracts value flows from variable declarations and assignment expressions. Literal assignments are recorded in the constant-value component V_c , whereas assignments whose right-hand side is an identifier, member path, call expression, binary expression, or template literal are recorded as symbolic dependencies in the mutable-binding component V_m . Instead of immediately resolving these dependencies, *JScamd* preserves them as analysis facts so that aliases, object paths, and derived values can be resolved during later taint propagation. This design is necessary because cryptographic APIs in JavaScript are often invoked indirectly. As shown in Figure 3, `const enc = crypto.createCipheriv` at

Line 8 does not define a new cryptographic operation; it creates an alias from `enc` to the normalized API path `crypto.createCipheriv`. By recording this relation in V_m , JScamd can later recognize `enc(...)` as an indirect call to `crypto.createCipheriv(...)`. This enables the analysis to capture function aliases without relying only on syntactic API names at call sites.

Step 2: Path normalization for API references. The second step normalizes `MemberExpression` nodes into canonical API paths. Rather than preserving nested member accesses as raw AST subtrees, JScamd recursively expands them into explicit paths, such as `crypto.subtle.encrypt` or `this.ctx.cipher.update`. This makes cryptographic API references explicit even when they are reached through library namespaces, object chains, or object fields. For computed properties, JScamd resolves the property name when it can be statically evaluated from string literals, constant bindings, template literals, or bounded expressions. If the property cannot be resolved statically, JScamd retains it as a symbolic property dependency for later propagation. This preserves dynamic access information without forcing unsound resolution.

Step 3: Function summaries for inter-procedural flows. The third step builds lightweight function summaries for inter-procedural propagation. For each function declaration or function expression, JScamd records its name, formal parameters, return expressions, source location, and lexical scope in F . Return expressions may be literals, identifiers, member paths, object fields, or call results. When a function call result is assigned to a variable or object field, JScamd links the receiving binding to the callee’s return summary whenever the callee can be statically resolved. This design is important because security-sensitive values are often computed in helper functions before reaching cryptographic sinks. For instance, an IV may be initialized as a constant as shown at Line 2 in Figure 3, returned by `getIV()` at Line 5, stored in the object field `opts.iv` at Line 10, copied into the local variable `iv` at Line 13, and eventually passed to the third argument of `createCipheriv` at Line 14. Function summaries make these cross-function dependencies explicit in the IR, allowing the taint engine to recover flows that are not visible from the sink call alone.

Step 4: Materializing call sites and object fields. For each call expression or constructor expression, JScamd creates a call-site record in C , including the normalized callee, ordered arguments, source location, and lexical scope. The ordered argument list is important because many cryptographic rules are position-sensitive. JScamd further represents configuration objects using field-sensitive facts. Many JavaScript cryptographic APIs, such as the Web Cryptography API, pass security-relevant parameters through configuration objects rather than positional arguments. Accordingly, JScamd extracts object literals and property assignments into explicit field records in O . For example, JScamd represents the object literal `{name: "AES-CBC", iv: x}` at Line 10 of Figure 3

as two field-level IR records, one for `name` and the other for `iv`. This field-sensitive representation allows JScamd to distinguish algorithm names, IVs, salts, iteration counts, keys, and certificate-validation flags even when they appear in the same configuration object. Finally, JScamd associates each call-site and field-level record with its lexical scope. During dependency resolution, JScamd only considers definitions that are visible at the corresponding use site. This scope-aware design prevents arguments or object fields from being linked to unrelated definitions, thereby reducing spurious data-flow propagation.

4.3 LLM-assisted Taint Analysis Rule Generation Pipeline

Traditional static taint analysis frameworks rely heavily on manually written taint rules, each describing specific $\langle \text{source}, \text{sink}, \text{filter} \rangle$ relationships. Such manual efforts are not only time-consuming but also incomplete, as new cryptographic APIs and misuse scenarios continuously emerge. To overcome this limitation, JScamd introduces an LLM-assisted rule extraction and source/sink labeling framework that automatically transforms natural-language misuse descriptions into deployable static taint rules consistent with JScamd internal rule schema.

The framework aims to automatically generate structured taint analysis rules for cryptographic API misuse detection by combining library-specific knowledge, natural-language misuse definitions, and static analysis constraints. Given the source code or documentation of a JavaScript cryptographic library, the LLM extracts all available API definitions, infers their functional categories, identifies security-relevant parameters, and finally outputs rule objects directly executable by the JScamd engine. In this way, JScamd can be extended to new cryptographic libraries without manual engineering effort, ensuring scalability and adaptability.

Our experiments reveal two primary difficulties in LLM-assisted rule extraction. First, large models often lack deep understanding of specific cryptographic APIs, occasionally hallucinating unsupported modes (e.g., “AES-OFB”) or mislabeling parameter semantics. Second, the models are unaware of the static analyzer’s internal logic, sometimes generating unrecognized fields (e.g., nonexistent judge functions). We address these through prompt-level alignment—explicitly encoding analyzer constraints, white-listed APIs, and validated examples—allowing the model to generate executable and verifiable taint rules that integrate seamlessly with JScamd.

The overall workflow of the LLM-assisted rule generation framework consists of three main stages:

(1) API Extraction And Preprocessing. Using the AST construction module, JScamd parses each cryptographic library’s source code to extract candidate APIs. JavaScript libraries typically include both user-facing functions (e.g., `subtle.encrypt`, `AES.encrypt`) and numerous internal

helpers (e.g., `_deriveBytes`, `toString`). To ensure analysis precision, we filter out irrelevant APIs through keyword-based heuristics (e.g., `encrypt`, `hash`, `sign`, `derive`, `iv`) and semantic matching of parameter lists. The resulting API candidate list forms the foundation for subsequent reasoning.

(2) LLM-assisted API Semantic Mapping. Each API candidate is paired with a corresponding natural-language misuse rule and minimal contextual knowledge (e.g., the API’s function description and parameter names). The LLM receives this structured input and determines whether the API semantically matches the misuse description. The detailed prompt design and decision workflow are illustrated in Figure 6 in the Appendix A.2. This classification process transforms descriptive security constraints into API and parameter associations that form the basis for taint rule instantiation.

(3) Rule Construction And Labeling. After identifying relevant API–rule pairs, the model instantiates formal taint rules using *JScamd*’s internal `rule()` constructor. To ensure that the LLM-generated taint-analysis rules are semantically accurate and executable in *JScamd*, we design a structured prompt template that guides the model’s reasoning and constrains its generation space. As shown in Figure 7 in the Appendix A.2, the prompt consists of four main sections: the task instruction, knowledge context, framework constraint, and one-shot demonstration. Each section provides distinct yet complementary information that guides the LLM to generate accurate and executable taint analysis rules for cryptographic API misuse detection.

The task instruction specifies the overall objective and output format, instructing the model to produce a JavaScript `rule` object compatible with *JScamd* static analysis engine. The knowledge context incorporates the natural-language description of the misuse rule and relevant information from the cryptographic library’s API, including function names, parameter descriptions, and algorithm categories. For instance, when generating a rule for the misuse “Do not use constant IV in AES-CBC mode”, this section provides the model with the API signature `subtle.encrypt(algorithm, key, data)` and identifies `iv` as the security-sensitive parameter.

The framework constraint section encodes the operational semantics of *JScamd* analyzer, including supported fields, taint sources, sinks, and judgment functions. This ensures that the model’s output adheres to the syntactic and semantic conventions of the taint engine, thus avoiding incompatible or non-executable rules. The one-shot demonstration section then provides one or two verified examples of correct rule definitions, allowing the model to infer the expected structure and stylistic conventions.

When constructing the prompt, we also dynamically adjust the content length to maintain efficiency. If the library documentation or API list is excessively long, we retain only the subset of APIs directly related to the targeted misuse category (e.g., encryption, hashing, and key derivation) to avoid exceeding model context limits.

We expect the LLM to respond in structured JSON format, where each field of the `rule` object is instantiated according to the analyzer’s schema, as shown in Figure 4. The model is instructed to produce both the rule definition and an accompanying brief explanation describing its reasoning, as this “generate-then-justify” pattern consistently produces more stable and interpretable outputs. Furthermore, when the model determines that a misuse rule does not apply to the given API, it explicitly indicates the reason (e.g., “unsupported algorithm” or “no related parameter”), which allows *JScamd* to prune irrelevant mappings and reduce redundant LLM calls. This design allows the model to generate rules that are both syntactically compliant and semantically aligned with real cryptographic API behaviors, thereby bridging the gap between high-level natural-language misuse descriptions and executable static analysis constraints.

```
1 const rule_InsecureIVWeb = new rule(  
2   ['*CONSTOBJ*'],  
3   ['subtle-encrypt'],  
4   ['getRandomValues'],  
5   [0],  
6   "using hard-coded IV in AES-CBC encryption",  
7   "C-05",  
8   [2],  
9   ['iv'],  
10  "replay-attack",  
11  "Medium",  
12  "const"  
13 );
```

Figure 4: An example output generated from the structured prompt.

By combining real API documentation, formal schema constraints, and in-context examples, the framework achieves rule outputs that are semantically aligned with cryptographic standards and syntactically compatible with *JScamd*.

After rule generation, *JScamd* executes automatic consistency checks and small-scale evaluation. Any mismatch—such as nonexistent sinks or invalid `judgefun`—is flagged and returned to the LLM for self-correction. This verification loop forms a human–AI hybrid refinement pipeline: the model provides rule drafts, the static analyzer validates their feasibility, and developers optionally refine semantics for ambiguous cases.

The final verified rules are stored in a unified `RuleMap` repository for the static taint engine. This allows direct invocation during inter-procedural and intra-procedural propagation, enabling automated misuse detection. As new rules or APIs emerge, the LLM-assisted pipeline can be re-invoked to incrementally expand the rule base, ensuring that *JScamd* remains adaptive and continuously up-to-date with the evolving JavaScript cryptographic ecosystem.

4.4 Precise Static Taint Analysis Engine

To enable context-aware detection of cryptographic API misuse in JavaScript, *JSscamd* employs a static taint analysis engine that combines rule-guided reasoning, dynamically constructed propagation paths, and fine-grained taint tracking. Unlike conventional approaches based on fixed pattern matching or shallow flow inspection, our engine couples security rule semantics with program-level data dependencies, enabling precise reasoning about multi-stage cryptographic usage.

(1) Rule Mapping And Dynamic Path Construction.

Once the element lists are established (functions, calls, variables, constants, and objects), *JSscamd* initializes rule mapping by loading corresponding taint rules derived from the rule repository. Each rule defines the triplet $\langle source, sink, filter \rangle$ and specifies the expected flow conditions. For instance, the misuse rule C-04 forbids the use of a constant IV in block cipher modes. The constant `iv` is initialized by a deterministic buffer as the source; the call to `crypto.createCipheriv()` represents a cryptographic sink; and no randomization filter such as `crypto.randomBytes()` exists in the propagation path. Consequently, the taint engine records the variable `iv` in the blacklist and reports a violation of rule C-04.

(2) Propagation Tracking And Blacklist Recording.

During analysis, *JSscamd* dynamically constructs both inter and intra propagation paths by linking element nodes through AST indices. Each intermediate propagation is logged in auxiliary structures, including: *Propagation Tables* for recording cross-scope variable dependencies; *Blacklists* for marking insecure or unfiltered taint elements; *Context Maps* for correlating call expressions with their lexical environments. The engine continuously updates these lists during traversal to ensure that any unfiltered data flow from insecure sources to cryptographic sinks can be precisely reported. This fine-grained propagation modeling enables *JSscamd* to detect misuse patterns that are typically invisible to conventional static analysis tools limited by JavaScript's dynamic nature.

(3) Four Kinds of Taint Analysis Frameworks.

Based on cryptographic interface design and common calling scenarios, we design four complementary taint analysis frameworks.

Single-Point Identification Framework. This framework directly identifies unsafe points by detecting parameters or cryptographic methods that violate security standards. Once identified, a misuse alert is generated. When the risk point is clearly defined and can be easily located in the AST, this approach provides fast and efficient detection. For example, in CryptoJS and SJCL, symmetric algorithms are implemented as classes and encryption operations are invoked through specific algorithm classes such as AES or DES. According to rule C-01, DES is considered insecure for practical use; therefore, any DES invocation can be directly classified as misuse. Similarly, weak hash algorithms such as RIPEMD-160 and SHA-1 can be directly flagged as security risks.

Forward Taint-Tracking Framework. The forward taint-

tracking framework analyzes the propagation path from a source to a sink. It is widely used in static analysis and is particularly suitable for scenarios where pollution sources can be explicitly identified. It requires that risky variables or method calls possess distinctive characteristics and are easy to locate. In Node.js Crypto, encryption functions targeted by rules for insecure algorithms (C-01), ECB-mode encryption (C-03), non-random IVs (C-04), and others commonly invoke `createCipheriv()`. The selection of algorithms, modes, and other parameters is performed via method arguments. For misuse detection, we first identify variables associated with the algorithm or working mode, and then analyze the information flow in the AST to determine whether the tainted source propagates to the sink point. This forward analysis traces tainted information and exposes potentially unsafe configurations or misuse instances.

Backward Taint-Tracking Framework. The backward taint-tracking framework analyzes propagation starting from the sink to identify the corresponding source. It is primarily used when the source cannot be explicitly defined but can be inferred from indications provided by the sink. Implementing rules such as C-01 and C-09 is challenging because their sources are often common values without clear distinguishing features. For such detections, we begin from an aggregation point. We first identify APIs that pose aggregation risks, such as key derivation functions or key generation functions, and then trace backward to recover the original value that was passed in. The traced parameter can correspond to either the argument position targeted by the analysis or a key value within an option structure.

Hybrid Taint-Tracking Framework. The hybrid framework combines forward and backward analysis and is designed to handle scenarios involving multi-directional dependencies, where multiple pollution sources may traverse the same sink. The design is driven by two practical detection requirements. First, it supports randomness assessment of IVs in CBC and CFB modes, where the two pollution sources to be located are the block cipher working mode and the IV. Second, it addresses security-flag assessments for cookies that store sensitive information, where the two pollution sources to be located are the sensitive data itself (e.g., JWT or session ID) and the security flag (e.g., `HttpOnly`). The core idea is to first use one pollution source to determine whether a risk scenario is present; if a potential risk is established, the analysis then searches for another pollution source to confirm whether an actual misuse has occurred.

5 Evaluation

In this section, we describe the implementation of *JSscamd* and present a comparative analysis of its detection results. Our evaluation is conducted on a large set of open-source JavaScript repositories.

5.1 Experimental Setup

JScamd was developed and implemented using JavaScript. Drawing inspiration from the JsPrime [33] taint analysis concept, we constructed a detection solution specifically targeting cryptographic API misuses. To facilitate this analysis, we utilized the Esprima [21] to convert JavaScript code into an intermediate representation.

The experiment is conducted on a 12th Gen Intel(R) Core(TM) i7-12700H 2.30 GHz with 16GB of RAM. On this platform, *JScamd* analyzes JavaScript code at approximately 1.55 KLOC/s, with a peak memory usage of 460 MB. To evaluate the effectiveness of *JScamd*, we constructed a comprehensive dataset derived from open-source projects hosted on GitHub and the npm ecosystem. The dataset was curated through a two-stage selection process: (1) Famous Open-Source Frameworks. Projects were first selected based on their popularity and recognition within the open-source community. We ranked repositories by metrics such as the number of stars and forks, and selected representative frameworks implemented in JavaScript and Node.js, including *React*, *Bootstrap*, *Electron*, and *VS Code*. These projects are widely adopted in both industrial and academic contexts, ensuring that our evaluation reflects real-world software development practices. (2) Keyword-Based Screening. To expand coverage to security- and cryptography-related projects, we performed keyword-based filtering using cryptographic-related terms. Additionally, repositories associated with cryptographic functionalities, such as Key Management Systems (KMS), Public Key Infrastructure (PKI), and encryption utilities, were included.

After integration and manual verification, a total of 322 JavaScript repositories were selected to form the final dataset. Among them, the most popular repository accumulated 411k stars and 118.7k forks, demonstrating a broad and diverse code base representative of real-world cryptographic usage in modern JavaScript ecosystems.

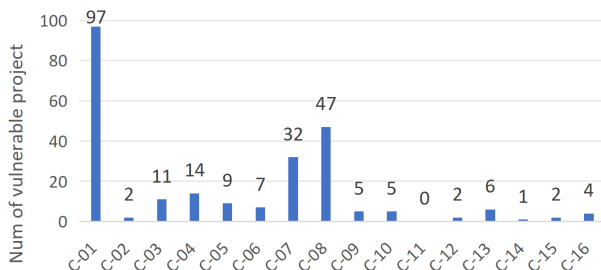


Figure 5: Number of vulnerable projects detected for each cryptographic misuse constraint.

5.2 Evaluation Detection Result

JScamd detected 322 JavaScript projects in the data set and found a total of 1083 instances of cryptographic API misuse,

of which 210 projects exposed at least one cryptographic API misuse, accounting for approximately 65.22% of the total number of projects. After detailed manual comparison, 1059 of all misuse prompts were confirmed to be true positive alerts, and the overall detection precision of the *JScamd* tool was 97.78%. It was identified that 24 of the prompts were false positives. These false positives were primarily caused by confusion in detecting unofficial APIs with the same name, such as misidentifying the program self-implemented function *sign* as the standard signature method *crypto.sign*.

Table 3 summarizes the overall experimental results and Figure 5 illustrates the distribution of vulnerable projects across different cryptographic misuse constraints. To further investigate cryptographic API misuse in real-world JavaScript security applications, we analyze the identified vulnerabilities along the six dimensions defined in Section 3.2.

Cryptographic Algorithm Security. According to Table 4, the weak hash algorithm associated with C-01-d causes the most problems, specifically the incorrect use of MD5 and SHA-1 algorithms. This issue affects 73 projects. The vulnerabilities identified by C-01-a detection primarily revolve around the utilization of outdated algorithms such as RC4 and DES. C-01-b and C-01-e detected the fewest problems, with 25 and 4 cases respectively. In addition, only one misuse alert was generated in the C-01-c detection. The main reason is that the existing API supports default safe elliptic curve parameters, and the possibility of developers specifically selecting unsafe parameters when calling this method is very low. In the detection of C-02, 2 applications were found to abuse *modp2* with insufficient key length.

Cipher Mode Security. There are 56 misuse cases of cipher mode security problems reported by *JScamd*. The ECB-mode constraint (C-03) has 16 misuse alerts, the open-source programs expose several instances of using the ECB mode block cipher algorithm. For example, *tuyapi* (stars 2k, forks 326) utilizes AES-ECB, provided by the cipher interface library, as the default encryption method within the protocol. *Stf* (stars 2.4k, forks 403) employs DES-ECB to generate encrypted responses. These usages undermine the security of the encrypted data and compromise its confidentiality and integrity. Using hard-coded IVs in CBC encryption mode is a common misuse problem and violates both C-04 and C-05. This misuse has been identified in the *muban* (stars 175, forks 12). Furthermore, it should be noted that the test code for *deno* (92.5k, 5.1k) and *CaptfEncoder* (1.1k, 196) still relies on the *PKCS1-v1.5* padding mode, which can pose a risk of padding oracle attacks.

Key Lifecycle Security. The detection constraints include C-07 and C-08, which emphasize the randomness of the key salt value, and C-09 and C-10, which center around the key derivation method. Among these constraints, C-07 and C-08 trigger more alarms, with 283 and 135 occurrences respectively. These results highlight the prevalence of concerns related to predictable keys and salt values, indicating a lack

Table 3: Cryptographic API misuse detection results and performance comparison of *JScamd* and state-of-the-art tools. TP denotes true positives, and FP denotes false positives. Semgrep* indicates Semgrep integrated with additional rules synthesized by the LLM-assisted rule generation pipeline.

ID	JScamd					State-of-the-Art Tools							
	Alerted Project	Alerts	TP	FP	Precision	Semgrep		Njsscan		CodeQL		Semgrep*	
						TP	FP	TP	FP	TP	FP	TP	FP
C-01	97 (46.19%)	499	499 (47.12%)	0	100.00%	19	0	280	0	31	0	472	0
C-02	2 (0.95%)	6	6 (0.57%)	0	100.00%	-	-	-	-	-	-	6	0
C-03	11 (5.24%)	16	16 (1.51%)	0	100.00%	13	6	16	2	-	-	-	-
C-04	14 (6.67%)	20	20 (1.89%)	0	100.00%	7	48	-	-	-	-	-	-
C-05	9 (4.29%)	20	20 (1.89%)	0	100.00%	8	6	4	2	-	-	-	-
C-06	7 (3.33%)	14	14 (1.32%)	0	100.00%	-	-	-	-	-	-	14	0
C-07	32 (15.24%)	307	283 (26.72%)	24	92.18%	1	0	99	21	20	19	-	-
C-08	47 (22.38%)	135	135 (12.75%)	0	100.00%	-	-	-	-	-	-	81	3
C-09	5 (2.38%)	10	10 (0.94%)	0	100.00%	-	-	-	-	-	-	10	0
C-10	5 (2.38%)	10	10 (0.94%)	0	100.00%	-	-	-	-	-	-	10	0
C-11	0 (0.00%)	0	0 (0.00%)	0	0.00%	0	9	0	119	0	97	-	-
C-12	2 (0.95%)	2	2 (0.19%)	0	100.00%	0	0	-	-	-	-	-	-
C-13	6 (2.86%)	36	36 (3.40%)	0	100.00%	36	4	0	10	36	14	-	-
C-14	1 (0.48%)	1	1 (0.09%)	0	100.00%	0	0	0	0	-	-	-	-
C-15	2 (0.95%)	2	2 (0.19%)	0	100.00%	-	-	-	-	-	-	2	0
C-16	4 (1.90%)	5	5 (0.47%)	0	100.00%	2	4	2	0	-	-	-	-
Total	210	1083	1059	24	97.78%	86	77	401	154	87	130	123	3

Table 4: Detection results for the sub-constraints of C-01, covering insecure cryptographic algorithms across symmetric encryption, asymmetric encryption, elliptic-curve parameters, hash functions, and HMAC constructions.

ID	Alerted Projects	TP
C-01-a	17	43 (8.62%)
C-01-b	3	25 (5.01%)
C-01-c	1	1 (0.20%)
C-01-d	73	426 (85.37%)
C-01-e	3	4 (0.80%)
Total	79	499 (100.00%)

of understanding among developers regarding key protection and the use of random salt values in key derivation. DoraCMS (stars 3.4k, forks 1k) generates 23 alerts about C-07. These alerts are triggered due to the usage of a common cryptographic algorithm implemented in *crypto.js*, where the hard-coded key *AESkey* is pinned to the AES encryption method for CryptoJS. Twake (stars 1.7k, forks 185) has revealed 2 issues in both C-09 and C-10. The number of iterations using the PBKDF2 method to derive keys is 9, which is seriously less than the security practice requirements.

Random Number Security. We detect no insecure PRNG

usage (C-11). Modern JavaScript cryptographic libraries expose secure PRNG interfaces by default, and official documentation of Node.js Crypto and the Web Cryptography API explicitly mandates secure random number generation. Moreover, *JScamd* restricts PRNG checks to security-critical contexts, treating statistical PRNGs as misuse only when used in cryptographic operations. This design reduces false positives reported in prior work [10, 39, 51].

Cryptographic Protocol Security. There are a total of 39 cases of protocol security issues discovered by *JScamd*. The majority correspond to violations of C-13, where 36 applications failed to verify certificate validity during network connections. C-12 screened 2 vulnerabilities using insecure HTTP parsers. During the detection of C-14, only one application was found to be using insecure cryptographic protocols.

Paastis is an HTTP proxy for hosting PaaS applications. Under rule C-12, *JScamd* identifies that Paastis enables the `insecureHTTPParser` option when calling `http.createServer()`. While intended to accommodate non-standard requests, this setting may trigger parser errors or unexpected behavior. MeshCentral exposes six misuse instances under C-13 by setting `rejectUnauthorized` to `false` when creating server instances, allowing certificates to bypass verification and enabling potential man-in-the-middle attacks. It also violates C-14 by setting `minVersion` to `TLSv1` to maintain compatibility with legacy clients, despite known

vulnerabilities in TLSv1.

Secure Storage of Sensitive Parameters. *JScamd* detected 7 cases of sensitive parameter storage security issues. There are 2 misuses of randomly storing JWT without setting the security flag in the Cookie, which violates C-15. During the security screening on the storage of sensitive session parameters, it was found that there were 5 cases of program violations that stored session ID in Cookie without security protection.

In the server-side login verification logic of *react-dashboard* (1.5k stars, 364 forks), *JScamd* identifies a misuse corresponding to C-15. In *qbit-matUI* (321 stars, 26 forks), session IDs are stored in cookies accessible to client-side JavaScript, violating C-16. This configuration exposes session tokens to cross-site scripting attacks. Secure practice mandates marking sensitive cookies as `HttpOnly` to prevent client-side access.

5.3 Comparison with Existing Tools

We evaluate *JScamd* against three state-of-the-art tools: Semgrep [22], Njsscan [1], and CodeQL [17]. Table 3 summarizes the results, from which we draw several observations.

First, *JScamd* demonstrates the most comprehensive coverage of cryptographic API misuse scenarios in JavaScript. In contrast, the three competing tools only address a limited subset of the categories listed in Table 1. Specifically, Semgrep supports a maximum of 10 detection constraints, while Njsscan and CodeQL cover even fewer, with only 8 and 4 constraints respectively. This limited constraint scope results in a large number of false negatives. For instance, while all three tools identify generic weak algorithms (C-01), they fail to detect the usage of insecure elliptic curves (C-01-c).

Secondly, *JScamd* demonstrates superior detection accuracy compared to the three baseline tools. As evidenced by the results, our tool achieves an overall precision rate of 97.78%, successfully attaining 100% precision in 13 out of the 16 evaluated constraint categories. This high precision stems from *JScamd*'s extensive support for diverse cryptographic libraries and the robustness of its detection logic. By leveraging an LLM-assisted rule generation process, *JScamd* covers a significantly wider range of third-party library APIs and complex algorithm variations than manually curated rulesets. For instance, while Semgrep is capable of detecting insecure RSA keys, its default rules are narrowly scoped to the standard node crypto and the cryptojs library. This limitation results in a high number of false negatives when developers employ other popular libraries. In contrast, the LLM-assisted approach of *JScamd* automatically identifies and generates taint analysis rules for these alternative APIs, thereby eliminating such blind spots.

Furthermore, *JScamd* effectively addresses the challenge of identifying APIs introduced via dynamic import mechanisms. Regarding precision, the few false positives produced by *JScamd* primarily originate from naming collisions with

user-defined or non-cryptographic methods. Moreover, *JScamd* effectively eliminates false positives caused by the lack of semantic context—a prevalent issue in competing tools—through its context-sensitive inter-procedural analysis. Taking the detection of insecure Random Number Generators (RNGs) as an example: *JScamd* verifies whether the generator output flows into a cryptographically relevant sink (e.g., Key or IV generation). In contrast, existing tools merely flag the invocation of the API regardless of its purpose. Our manual verification confirmed that the majority of insecure RNG usage flagged by other tools occurred in benign, non-cryptographic scenarios, such as the generation of identifiers.

5.4 Effectiveness of Enhanced Program Representation Engine

To better quantify the effectiveness of the enhanced program representation engine in *JScamd*, we compare it with Semgrep, Njsscan, and CodeQL on the Obuhersys dataset [4], as shown in Table 5. Obuhersys is a benchmark designed to evaluate cryptographic misuse detection under complex JavaScript program features. Unlike simple static samples that contain direct API calls and constant parameters, Obuhersys incorporates inter-procedural data propagation, context-sensitive calls, path-sensitive conditional branches, field-/object-sensitive data transfer, and combinations of multiple sensitivity requirements. These features make cryptographic misuses appear only after variables, function calls, object properties, wrapper functions, or conditional branches are resolved, thereby providing a more challenging setting for evaluating the semantic modeling capability of static analysis tools. The Obuhersys dataset is constructed based on the `Node.js Crypto` library. To ensure a fair comparison, we evaluate only the security constraints that are supported by all tools, including insecure cryptographic algorithms (C-01-a), insecure hash algorithms (C-01-d), ECB-mode encryption (C-03), insecure IV usage in block cipher modes (C-04/C-05), and weak PBKDF2 configurations (C-09/C-10). The results show that *JScamd* achieves the best overall balance between detecting true misuses and controlling false positives on this challenging dataset. Compared with Semgrep, *JScamd* substantially improves recall from 6.67% to 38.33% and F1 score from 11.76% to 46.46%, indicating that pattern-based matching alone is insufficient for capturing cryptographic misuses hidden behind JavaScript data propagation. Compared with Njsscan, *JScamd* detects slightly more true positives while producing fewer false positives, leading to higher precision, recall, and F1 score. Compared with CodeQL, *JScamd* achieves much higher recall and F1 score, although CodeQL obtains the highest precision. This result suggests that CodeQL is conservative in this setting and misses many true misuse cases, whereas *JScamd* better captures security-relevant data flows through its enhanced representation of aliases, object fields, function returns, and scope-sensitive propagation.

Table 5: Detection performance on the Obuhersys dataset under the same security constraints and taint analysis rules across *JScamd* and state-of-the-art tools.

Tools	TP	TN	FP	FN	Precision	Recall	F1 score
Semgrep	4	62	4	56	50.00%	6.67%	11.76%
Njsscan	22	46	20	38	52.38%	36.67%	43.14%
CodeQL	11	59	7	49	61.11%	18.33%	28.21%
JScamd	23	50	16	37	58.97%	38.33%	46.46%

We further analyze the sources of false negatives and false positives produced by *JScamd* on this dataset. The false negatives mainly arise from two factors. First, *JScamd* still has limited modeling of field propagation through object properties, class constructors, and static factory methods. To avoid introducing excessive false positives in complex object-aliasing and field-assignment scenarios, *JScamd* adopts a relatively conservative propagation strategy. As a result, when an insecure value is indirectly propagated to a cryptographic API through object fields, constructor-returned objects, or factory methods, the complete flow may not always be recovered. Second, *JScamd* does not yet perform precise path-sensitive analysis for branch conditions. Therefore, in samples where the misuse depends on branch reachability, *JScamd* may fail to determine whether an insecure value necessarily reaches a sensitive cryptographic call. These false negatives mainly reflect the inherent trade-offs of static analysis in field sensitivity, context sensitivity, and path sensitivity. They are not a *JScamd*-specific failure pattern; instead, they reflect a broader limitation shared by static analysis tools when analyzing dynamic JavaScript programs. We also examine the sources of false positives. False positives mainly occur in scenarios involving custom identity functions, factory methods, higher-order function wrappers, and complex path conditions. When string-based rules cannot fully resolve runtime values, *JScamd* may mistakenly interpret keywords appearing in variable or field names as actual cryptographic algorithm values. Despite these limitations, the overall results show that *JScamd* achieves the best recall and F1 score, suggesting that it improves the coverage of cryptographic misuse detection while maintaining a practical trade-off between false negatives and false positives, which is a common challenge for static analysis tools.

5.5 Effectiveness of LLM-assisted Rule Generation

High-quality taint analysis rules are the cornerstone of effective static vulnerability detection. In this work, we propose a novel framework for generating cryptographic misuse rules using LLMs. Our framework aims to assist static analysis tools, particularly those used to detect cryptographic vulnerabilities in JavaScript applications. To evaluate the effectiveness of our

Table 6: API semantic mapping performance across models.

Model	Precision	Recall	F1 score
GPT-5.5	98.96%	99.82%	99.39%
Gemini-3.1	96.98%	95.61%	96.29%
DeepSeek-V3.2	96.76%	94.20%	95.47%

proposed LLM-assisted rule generation pipeline, we focus on two core tasks: API semantic mapping and rule generation. We compare three widely used large language models with different architectural designs and capability levels: GPT-5.5, Gemini-3.1, and DeepSeek-V3.2. We further report the monetary cost of rule generation, which is \$4.63 for GPT-5.5, \$2.51 for Gemini-3.1, and \$0.12 for DeepSeek-V3.2. These results allow us to assess not only rule quality but also the cost and performance trade-offs of different models.

API Semantic Mapping. For the API semantic mapping task, we evaluate whether LLMs can map natural language cryptographic security rules to relevant API implementations. We perform the evaluation measuring the precision, recall and F1 score of each model to identify relevant APIs. For each rule, we evaluate whether the model successfully identified the relevant API and mapped the correct API parameters to match the description of the rule. As shown in Table 6, all tested models achieved high accuracy in correctly mapping APIs to their corresponding semantic meanings.

Rule Generation. To further evaluate whether our framework reduces LLM hallucinations during taint-rule generation, we conduct an evaluation on a benchmark consisting of 367 APIs from CryptoJS and SJCL with manually validated ground truths. In this experiment, we define a hallucinated rule as a generated rule that satisfies at least one of the following conditions: (1) it refers to non-existent, unsupported, or out-of-scope cryptographic APIs; (2) it uses judgment functions that are incompatible with the *JScamd* rule schema; or (3) it assigns incorrect API parameters or rule logic that violates the intended security constraint.

We compare two rule-generation configurations. The first is direct generation, where the LLM directly translates a natural-language security constraint into a *JScamd* taint rule. To make this baseline non-trivial, the prompt provides role instructions, library-specific API context, parameter positions,

configuration-object patterns, security-relevant keywords, expected data-flow relations, and the JScamd rule schema. The second configuration is JScamd’s LLM-assisted rule generation pipeline, which decomposes rule generation into API semantic mapping and rule instantiation.

As shown in Table 7, direct generation still suffers from substantial hallucinations across all evaluated models, even with the above prompt guidance. GPT-5.5, Gemini-3.1, and DeepSeek-V3.2 exhibit hallucination rates of 66.36%, 68.22%, and 46.97%, respectively. Manual inspection shows that hallucinations in the direct-generation setting mainly arise from two sources. First, the models often produce pseudo sources, sinks, or filters that do not correspond to APIs in the target cryptographic library. For example, when generating rules for CryptoJS, GPT-5.5 introduced `jsonwebtoken.sign` as a source and `res.cookie` as a sink, although JWT signing and cookie-setting API are outside the CryptoJS API surface. Similar errors are observed in Gemini’s rule generation results for SJCL. DeepSeek exhibits a lower hallucination rate under direct generation mainly because its outputs are more conservative and less expansive. In contrast, JScamd’s LLM-assisted pipeline substantially reduces hallucinations. The hallucination rates drop to 2.90% for GPT-5.5, 4.35% for Gemini-3.1, and 5.80% for DeepSeek-V3.2. The remaining hallucinations in JScamd mainly come from schema-level mismatches, especially cases where the model selects a judgment method that is not supported by the current JScamd implementation. This occurs because the LLM has only partial knowledge of the internal semantics of JScamd’s rule interpreter. However, unlike direct-generation hallucinations, these residual errors are localized and mechanically checkable: the rules are still grounded in validated cryptographic APIs and misuse semantics, but fail at the final executable-schema layer.

Table 7: Hallucination rates of direct LLM rule generation and JScamd’s constrained rule-generation pipeline on 367 manually validated CryptoJS and SJCL APIs with manually validated ground truths.

Model	Direct Generation	JScamd
GPT-5.5	66.36%	2.90%
Gemini-3.1	68.22%	4.35%
DeepSeek-V3.2	46.97%	5.80%

Generalizability and Extensibility. To evaluate the versatility of our framework, we extend it to generate Semgrep rules. As shown in Table 3, our LLM-assisted pipeline successfully filled gap in Semgrep cryptographic misuse detection. The effectiveness of rule generation is influenced by the complexity of the target rule language. Because Semgrep adopts an intuitive YAML-based pattern syntax, the LLM could reliably translate natural-language security constraints into ex-

ecutable detection rules, achieving coverage comparable to manually written rules. In contrast, automatic CodeQL query generation showed limited effectiveness. The strict typing, object-oriented predicates, and complex data-flow semantics of QL made it difficult for the LLM to consistently produce correct queries without manual intervention.

6 Limitation and Discussion

Similar to prior work [10, 26, 39], *JScamd* has limitations that warrant future improvement.

Detection Accuracy. Most observed false positives stem from ambiguities caused by APIs sharing identical names. Since *JScamd* focuses on identifying key APIs and parameters rather than inferring concrete algorithmic semantics, it may misinterpret similarly named functions or orthogonal calls, as also noted in prior work such as CryptoGuard [39] and CryptoGo [26]. In response to the false-positive concerns raised by Chen et al. [10], we attempt to constrain the security context when defining rules (e.g., treating insecure PRNGs as misuse only when used for security-critical cryptographic operations). However, existing tools still struggle to precisely determine the security intent of API usage. In addition, false negatives may occur when security-sensitive values are propagated through complex JavaScript features, such as object fields, factory methods, higher-order wrappers, or path-dependent branches. These cases require more precise field-, context-, or path-sensitive reasoning, which remains challenging for static analysis tools. Future integration of machine-learning-based semantic understanding may further reduce false positives and false negatives.

Language and Transformation Scope. JScamd operates on standard JavaScript and does not natively support TypeScript. Because TypeScript introduces extended syntax (e.g., type annotations and generic declarations), its raw source files cannot be directly processed by our current Esprima-based AST module. However, this is an engineering limitation of the frontend rather than a flaw in our taint analysis design. Future iterations can resolve this by integrating TypeScript-aware parsers. Second, JScamd does not currently handle heavily minified or obfuscated JavaScript code. These transformation techniques intentionally destroy formatting, flatten control flows, and obscure data dependencies. Such structural alterations hinder JScamd from accurately constructing taint propagation paths. Consequently, our current evaluation focuses on source-level repositories. Handling transformed code would require dedicated preprocessing (e.g., source-map recovery and de-obfuscation), which is left for future work.

LLM Dependence and Model Evolution. *JScamd* relies on large language models for semantic rule generation. As foundation models rapidly advanced, we revisited our pipeline with more recent LLMs and observed noticeable improvements in semantic coherence, stability in API-misuse mapping, and correctness in taint rule instantiation. These

observations suggest that our approach is inherently model-agnostic and can benefit from continual progress in LLM technology. It is also worth noting that all quantitative experimental results reported in this paper are based on the most recent dataset collected in December 2025, ensuring that the evaluation reflects contemporary cryptographic library implementations and practical misuse patterns.

LLM-agent Based Analysis. We also conducted a supplementary evaluation using an LLM-agent. Specifically, we applied Codex [35] to analyze the real-world project MeshCentral, which contains 132 JavaScript files and approximately 107,390 lines of JavaScript code. The agent-based analysis cost \$2.3 for this single project, yet it identified only 13 of the 42 true positives detected by JScamd. This result suggests that directly applying LLM agents to repository-level cryptographic misuse detection remains less effective and less cost-efficient than JScamd rule-guided static analysis.

Responsible Disclosure. Based on the findings summarized in Table 3, we contacted developers of affected projects and followed responsible disclosure practices. Among 124 contacted projects, maintainers of 23 projects provided substantive responses as of May 2026. Specifically, 12 projects confirmed the reported issues, and some have implemented fixes or security improvements; for example, bcoin upgraded its hashing algorithm to SHA-256. Other maintainers argued that weak cryptographic primitives were used in non-security contexts (e.g., meteor, whistle), or that insecure designs were constrained by legacy protocols, reverse-engineering requirements, or deployment assumptions (e.g., tuyapi, ioBroker.mihome, MeshCentral). The disclosure process reflects a common gap between cryptographic misuse patterns and developers' perception of practical exploitability. Similar response patterns have also been observed in prior cryptographic misuse studies, where confirmation and remediation rates were limited despite technically valid reports [39]. In total, we obtained 41 CVE IDs, 21 of which are currently under embargo and have not yet been publicly disclosed.

7 Related Work

Cryptographic API Misuse Detection. Numerous studies have demonstrated that cryptographic API misuse is pervasive across real-world software ecosystems. Early research predominantly focused on Java-based applications, where tools such as CryptoLint [14], CryptoGuard [39], JWTKey [50] and Crylogger [36] systematically analyzed Java Cryptography Architecture (JCA) usage to detect cryptographic API misuse using static or dynamic analysis techniques. These works revealed that developers frequently misuse cryptographic APIs due to complex interfaces, insufficient guidance, or misunderstanding of security guarantees. Subsequent research efforts extended cryptographic API misuse detection beyond Java to a broader range of languages and platforms, consistently confirming that the problem is not language-specific but rather a

widespread software engineering challenge. Empirical analyses have identified large-scale cryptographic API misuses in C/C++ [51], Python [19, 47], and Go [26, 53], with a significant proportion of projects exhibiting issues. In recent years, the industry has also introduced multi-language static analysis tools such as CodeQL [17] and Semgrep [22], which provide cryptography-related detection rules. However, these general-purpose multilingual tools often struggle to handle language-specific characteristics effectively and lack domain-specialized cryptographic detection capabilities.

JavaScript Static Analysis and Security Detection. JavaScript static analysis has been widely studied. However, the features of the JavaScript language make it difficult to precisely recover control flow, call relationships, and data dependencies [18]. Flow [9] demonstrates that scalable static type checking can be deployed on large JavaScript code bases. The event-driven analysis for Node.js [32] applications and modular call-graph construction address callback relationships, module boundaries, and cross-package analysis in server-side JavaScript. Security-oriented JavaScript analysis commonly relies on taint tracking and dependence modeling. Wei et al. [45] combine runtime information, such as object types, dynamically generated code, and execution traces, with static taint analysis to mitigate the imprecision and timeout problems of purely static JavaScript analysis. TAJs/TAJstaint-based work [3] integrates abstract interpretation, string automata, and rendering-context-aware sanitization to detect DOM-based XSS. More recently, ODGen [25] proposes an object dependence graph that uniformly models JavaScript objects, AST nodes, control-flow dependencies, and data dependencies, and uses graph queries to detect Node.js vulnerabilities such as command injection, path traversal, and prototype pollution. Several studies are more closely related to JavaScript API misuse detection and taint-model construction. SAFEWAPI [5] detects Web API misuses using predefined misuse patterns, demonstrating the feasibility of rule-based API misuse detection in web applications. Staicu et al. [42] automatically extract taint specifications, including sources, sinks, and propagators, for JavaScript libraries to reduce the manual effort of maintaining taint models.

Existing studies focus heavily on low-level program semantics (e.g., types and taints) but lack a rule-facing abstraction that directly exposes security-sensitive API contexts. JScamd bridges this gap with a security-oriented IR that distills volatile dynamic patterns into stable semantic records, enabling direct and robust misuse detection

LLM-Assisted Security Analysis. Meanwhile, Large Language Models (LLMs) have achieved remarkable success in various real-world security analysis tasks. Researchers have begun exploring the integration of LLMs with programmable analysis frameworks to address more complex and semantically rich problems [11, 27, 29, 49, 52]. IRIS [27] proposed an LLM-assisted static analysis framework that leverages CodeQL for program analysis while using LLMs to label APIs

and associate them with vulnerability patterns.

Cryptographic API Usability. There are also studies that focus on improving the usability of cryptographic libraries, examining how interface design, API complexity, documentation quality, and abstraction mechanisms influence developers' ability to correctly and securely apply cryptographic functionality [2, 8, 30, 41, 48, 54]. While these studies reveal important design flaws and usability barriers, they largely stop short of detecting or quantifying concrete misuse in large-scale real-world JavaScript systems.

Compared with the above work, *JScamd* fills an important gap by providing a specialized framework dedicated to analyzing cryptographic API misuse in JavaScript applications. By leveraging enhanced static taint analysis techniques and LLM-guided rule generation, *JScamd* assists developers in identifying and mitigating cryptographic vulnerabilities, thereby improving the security of JavaScript applications that rely on cryptographic operations.

8 Conclusion

In this paper, we present *JScamd*, an innovative framework for detecting cryptographic API misuse across the JavaScript ecosystem. *JScamd* addresses the challenge of detecting cryptographic misuses in JavaScript through static taint analysis by combining an LLM-assisted rule generation pipeline with a JavaScript-specific program representation and taint analysis engine. This design transforms natural-language cryptographic constraints into executable analysis rules while preserving security-relevant JavaScript semantics, enabling accurate taint propagation and misuse detection across dynamic language constructs. Through comparative evaluation against SOTA tools, we demonstrate the effectiveness of *JScamd*. Applying our framework to 322 popular JavaScript projects, we identify 1,059 real-world misuse instances with a precision of 97.78%. Our findings provide the first comprehensive empirical understanding of cryptographic API misuse in the JavaScript community.

Acknowledgments

This work was supported by National Natural Science Foundation of China (No.62272457), National Cyber Security-National Science and Technology Major Project (No.2026ZD1500200), and Science and Technology Plan Project of Xizang Autonomous Region (No.XZ202502JD0008).

Ethical Considerations

Stakeholder Identification and Impacts. This research impacts JavaScript developers, security researchers, and end-users. By detecting cryptographic API misuses, *JScamd* di-

rectly helps developers remediate vulnerabilities and enhances end-user security. While automated detection tools can theoretically be exploited by malicious actors to locate weaknesses, our design and dissemination strictly prioritize defensive security and ecosystem reinforcement.

Cryptography Community and Society. Our study contributes to the broader cryptography and software security community by empirically characterizing misuse patterns in a widely deployed ecosystem. Publicly documenting these findings promotes transparency, reproducibility, and the development of safer APIs and analysis tools. At a societal level, improving the reliability of cryptographic implementations supports privacy, data protection, and trust in digital infrastructure. We align with community norms that favor responsible disclosure and open scientific communication to strengthen collective security.

Potential Harms Analysis. To mitigate the dual-use risk of our detection techniques, our analysis was strictly limited to public, open-source repositories without active exploitation, intrusive testing, or system modifications. Furthermore, we adhered to responsible disclosure by withholding vulnerability details and coordinating with maintainers prior to publication, minimizing immediate risks while preserving research utility.

Decision to Conduct and Publish. This study was undertaken to improve the robustness of cryptographic practices in the JavaScript ecosystem. The project was internally reviewed for technical merit and ethical considerations prior to execution. We adhered to responsible disclosure practices by reporting discovered issues to maintainers and coordinating CVE submissions when applicable (Section 6). We determined that the long-term security benefits of publishing outweigh the residual dual-use risks. Open dissemination encourages accountability, advances defensive tooling, and supports safer software development. The research was conducted solely for defensive purposes, and we did not identify ethical concerns sufficient to warrant withholding publication.

Open Science

We release *JScamd* along with the full experimental artifacts, including our analysis framework, dataset information, detection rules, execution scripts, configurations, and documentation required to reproduce all results reported in this paper. All shareable artifacts are available from a repository at <https://doi.org/10.5281/zenodo.20395867>. Due to licensing constraints, we do not redistribute third-party real-world JavaScript projects used in our evaluation.

References

- [1] Ajin Abraham. njsscan, 2023. URL: <https://github.com/ajinabraham/njsscan>.

- [2] Yasemin Acar, Michael Backes, Sascha Fahl, Simson L. Garfinkel, Doowon Kim, Michelle L. Mazurek, and Christian Stransky. Comparing the usability of cryptographic apis. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 154–171. IEEE Computer Society, 2017.
- [3] Nabil Almashfi and Lunjin Lu. Static taint analysis for javascript programs. In *International Conference on Tools and Methods for Program Analysis*, pages 155–167. Springer, 2019.
- [4] Ronak Badhe, Andrew Kuai, Leon Liu, Hunter Kang, Dylan Truong, Chinmay Devadhar, Pranav Akella, Kevin Xue, Anthony Fangqing Yu, and Teong Seng Tan. Obuhersys: Dynamic analysis of cryptographic api misuse in node. js. In *2024 IEEE MIT Undergraduate Research Technology Conference (URTC)*, pages 1–5. IEEE, 2024.
- [5] SungGyeong Bae, Hyunghun Cho, Inho Lim, and Sukyoung Ryu. Safewapi: Web api misuse detector for web applications. In *Proceedings of the 22nd ACM SIGSOFT international symposium on foundations of software engineering*, pages 507–517, 2014.
- [6] Elaine Barker, WC Barker, WE Burr, WT Polk, and ME Smid. Sp 800-57 part 1 rev. 5 recommendation for key management part 1: General. *NIST special publication*, 800:57, 2020.
- [7] Elaine Barker, William Barker, William Burr, William Polk, and Miles Smid. Nist special publication 800-57. *NIST Special publication*, 800(57):1–142, 2007.
- [8] Daniel J Bernstein, Tanja Lange, and Peter Schwabe. The security impact of a new cryptographic library. In *Progress in Cryptology–LATINCRYPT 2012: 2nd International Conference on Cryptology and Information Security in Latin America, Santiago, Chile, October 7-10, 2012. Proceedings 2*, pages 159–176. Springer, 2012.
- [9] Avik Chaudhuri, Panagiotis Vekris, Sam Goldman, Marshall Roch, and Gabriel Levi. Fast and precise type checking for javascript. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):1–30, 2017.
- [10] Yikang Chen, Yibo Liu, Ka Lok Wu, Duc V Le, and Sze Yiu Chau. Towards precise reporting of cryptographic misuses. In *Proceedings 2024 Network and Distributed System Security Symposium*, 2024.
- [11] Xueying Du, Geng Zheng, Kaixin Wang, Yi Zou, Yujia Wang, Wentai Deng, Jiayi Feng, Mingwei Liu, Bihuan Chen, Xin Peng, et al. Vul-rag: Enhancing llm-based vulnerability detection via knowledge-level rag. *arXiv preprint arXiv:2406.11147*, 2024.
- [12] M Dworkin. Recommendation for block cipher modes of operation. *Methods and Techniques*, 2001.
- [13] Morris J Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. Technical report, NIST Special Publication, 2007.
- [14] Manuel Egele, David Brumley, Yanick Fratantonio, and Christopher Kruegel. An empirical study of cryptographic misuse in android applications. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pages 73–84, 2013.
- [15] facebook. React, 2022. URL: <https://github.com/facebook/react>.
- [16] Felix Fischer, Konstantin Böttinger, Huang Xiao, Christian Stransky, Yasemin Acar, Michael Backes, and Sascha Fahl. Stack overflow considered harmful? the impact of copy&paste on android application security. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 121–136. IEEE, 2017.
- [17] Github. Codeql, 2021. URL: <https://github.com/github/codeql>.
- [18] Liang Gong. *Dynamic analysis for JavaScript code*. University of California, Berkeley, 2018.
- [19] Xiangxin Guo, Shijie Jia, Jingqiang Lin, Yuan Ma, Fangyu Zheng, Guangzheng Li, Bowen Xu, Yueqiang Cheng, and Kailiang Ji. Cryptopyt: Unraveling python cryptographic apis misuse with precise static taint analysis. In *2024 Annual Computer Security Applications Conference (ACSAC)*, pages 1075–1091. IEEE, 2024.
- [20] Harry Halpin. The w3c web cryptography api: motivation and overview. In *Proceedings of the 23rd International Conference on World Wide Web*, pages 959–964, 2014.
- [21] Ariya Hidayat. Esprima. <https://esprima.org/>, 2021.
- [22] Semgrep Inc. Semgrep. <https://semgrep.dev/>, 2025.
- [23] Michael Jones. RFC 7518: JSON Web Algorithms (JWA). Technical report, Internet Engineering Task Force, 2015.
- [24] Michael Jones and Joe Hildebrand. RFC 7516: JSON Web Encryption (JWE). Technical report, Internet Engineering Task Force, 2015.
- [25] Song Li, Mingqing Kang, Jianwei Hou, and Yinzhi Cao. Mining node.js vulnerabilities via object dependence graph and query. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 143–160, 2022.

- [26] Wenqing Li, Shijie Jia, Limin Liu, Fangyu Zheng, Yuan Ma, and Jingqiang Lin. Cryptogo: Automatic detection of go cryptographic api misuses. In *Proceedings of the 38th Annual Computer Security Applications Conference*, pages 318–331, 2022.
- [27] Ziyang Li, Saikat Dutta, and Mayur Naik. IRIS: IIm-assisted static analysis for detecting security vulnerabilities. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24–28, 2025*. OpenReview.net, 2025. URL: <https://openreview.net/forum?id=9LdJDU7E91>.
- [28] Fukang Liu, Christoph Dobraunig, Florian Mendel, Takanori Isobe, Gaoli Wang, and Zhenfu Cao. Efficient collision attack frameworks for ripemd-160. In *Annual International Cryptology Conference*, pages 117–149. Springer, 2019.
- [29] Peiyu Liu, Junming Liu, Lirong Fu, Kangjie Lu, Yifan Xia, Xuhong Zhang, Wenzhi Chen, Haiqin Weng, Shouling Ji, and Wenhai Wang. Exploring {ChatGPT’s} capabilities on vulnerability management. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 811–828, 2024.
- [30] Junwei Luo, Xun Yi, Fengling Han, and Xuechao Yang. A usability study of cryptographic api design. In *Quality, Reliability, Security and Robustness in Heterogeneous Systems: 17th EAI International Conference, QShine 2021, Virtual Event, November 29–30, 2021, Proceedings 17*, pages 194–213. Springer, 2021.
- [31] Jeff Mott. Crypto-js. <https://code.google.com/archive/p/crypto-js>, 2015.
- [32] Benjamin Barslev Nielsen, Martin Toldam Torp, and Anders Møller. Modular call graph construction for security scanning of node.js applications. In *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, pages 29–41, 2021.
- [33] Sarathi Sabyasachi Sahoo Nishant Das Patnaik. Jsprime. <https://github.com/dpnishant/jsprime>, 2015.
- [34] nodejs. Crypto. <https://nodejs.org/api/crypto.html>, 2009.
- [35] OpenAI. codex, 2026. URL: <https://openai.com/codex/>.
- [36] Luca Piccolboni, Giuseppe Di Guglielmo, Luca P Carloni, and Simha Sethumadhavan. Crylogger: Detecting crypto misuses dynamically. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1972–1989. IEEE, 2021.
- [37] Joe Gibbs Politz, Matthew J Carroll, Benjamin S Lerner, Justin Pombrio, and Shriram Krishnamurthi. A tested semantics for getters, setters, and eval in javascript. In *Proceedings of the 8th Symposium on Dynamic Languages*, pages 1–16, 2012.
- [38] Sazzadur Rahaman, Haipeng Cai, Omar Chowdhury, and Danfeng Yao. From theory to code: Identifying logical flaws in cryptographic implementations in c/c++. *IEEE transactions on dependable and secure computing*, 19(6):3790–3803, 2021.
- [39] Sazzadur Rahaman, Ya Xiao, Sharmin Afrose, Fahad Shaon, Ke Tian, Miles Frantz, Murat Kantarcioglu, and Danfeng Yao. Cryptoguard: High precision detection of cryptographic vulnerabilities in massive-sized java projects. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 2455–2472, 2019.
- [40] Ashutosh Satapathy, Jenila Livingston, et al. A comprehensive survey on ssl/tls and their vulnerabilities. *International Journal of Computer Applications*, 153(5):31–38, 2016.
- [41] Thomas Scheller and Eva Kühn. Automated measurement of API usability: The API concepts framework. *Information and Software Technology*, 61:145–162, 2015.
- [42] Cristian-Alexandru Staicu, Martin Toldam Torp, Max Schäfer, Anders Møller, and Michael Pradel. Extracting taint specifications for javascript libraries. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*, pages 198–209, 2020.
- [43] Emily Stark, Michael Hamburg, and Dan Boneh. Symmetric cryptography in javascript. In *2009 Annual Computer Security Applications Conference*, pages 373–381. IEEE, 2009.
- [44] Sean Turner and Lily Chen. Updated security considerations for the md5 message-digest and the hmac-md5 algorithms. Technical report, RFC Editor, 2011.
- [45] Shiyi Wei and Barbara G Ryder. Practical blended taint analysis for javascript. In *Proceedings of the 2013 International Symposium on Software Testing and Analysis*, pages 336–346, 2013.
- [46] Pascal Wichmann, Maximilian Blochberger, and Hannes Federrath. Web cryptography api: Prevalence and possible developer mistakes. In *Proceedings of the 17th International Conference on Availability, Reliability and Security*, pages 1–10, 2022.
- [47] Anna-Katharina Wickert, Lars Baumgärtner, Florian Breitfelder, and Mira Mezini. Python crypto misuses in

the wild. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 1–6, 2021.

- [48] Jan Wohlwender, Rolf Huesmann, Andreas Heinemann, and Alexander Wiesmaier. cryptolib: Comparing and selecting cryptography libraries. In *Proceedings of the 2022 European Interdisciplinary Cybersecurity Conference*, pages 6–11, 2022.
- [49] Yifan Xia, Zichen Xie, Peiyu Liu, Kangjie Lu, Yan Liu, Wenhai Wang, and Shouling Ji. Beyond static pattern matching? rethinking automatic cryptographic api misuse detection in the era of llms. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):113–136, 2025.
- [50] Bowen Xu, Shijie Jia, Jingqiang Lin, Fangyu Zheng, Yuan Ma, Limin Liu, Xiaozhuo Gu, and Li Song. Jwtkey: Automatic cryptographic vulnerability detection in jwt applications. In *European Symposium on Research in Computer Security*, pages 263–282. Springer, 2023.
- [51] Li Zhang, Jiongyi Chen, Wenrui Diao, Shanqing Guo, Jian Weng, and Kehuan Zhang. {CryptoREX}: large-scale analysis of cryptographic misuse in {IoT} devices. In *22nd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2019)*, pages 151–164, 2019.
- [52] XiangRui Zhang, XueJie Du, HaoYu Chen, Yongzhong He, Wenjia Niu, and Qiang Li. Automatically generating rules of malicious software packages via large language model. In *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 734–747. IEEE, 2025.
- [53] Yuexi Zhang, Bingyu Li, Jingqiang Lin, Linghui Li, Jiaju Bai, Shijie Jia, and Qianhong Wu. Gopher: High-precision and deep-dive detection of cryptographic api misuse in the go ecosystem. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 2978–2992, 2024.
- [54] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. Hacl*: A verified modern cryptographic library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, pages 1789–1806, 2017.

A Appendices

A.1 Details of Security Constraints

Cryptographic Algorithm Security. Secure cryptographic algorithms are fundamental to ensuring the confidentiality and integrity of security systems. According to NIST SP 800-57,

cryptographic mechanisms must provide sufficient security strength to withstand current and foreseeable computational capabilities, and the minimum required strength is expected to increase to 112 bits by 2030 [7]. *C-01* prohibits the use of weak or deprecated cryptographic algorithms. We further classify weak algorithms into several subclasses. *C-01-a* restricts the use of insecure symmetric encryption algorithms, such as DES and TDEA (Triple Data Encryption Algorithm). In particular, NIST mandates that TDEA must no longer be used for encryption after 2023 and may only be retained for backward-compatible decryption [6]; *C-01-b* concerns the secure use of asymmetric algorithms and prohibits key strengths below recommended thresholds, such as weak RSA or DSA parameters; *C-01-c* requires elliptic curve cryptography (ECC) to employ curves with a field size of at least 224 bits to ensure adequate security strength; *C-01-d* forbids the use of broken hashing algorithms that can no longer provide collision resistance; *C-01-e* prohibits insecure MAC constructions, such as HMAC-MD5, which are known to pose security risks. *C-02* prohibits the use of insecure Diffie–Hellman groups, including modp1, modp2, and modp5. This rule is also derived from security strength considerations. These DH groups provide key lengths of less than 2048 bits, significantly weakening the security guarantees of IPsec negotiations and other cryptographic protocols. Therefore, they should be replaced with stronger DH groups that meet modern security strength requirements.

Cipher Mode Security. The rules governing block cipher modes include *C-03*, *C-04*, and *C-05*. Rule *C-03* disallows the use of ECB mode, as identical plaintext blocks produce identical ciphertext blocks, leaking structural information and enabling pattern-based attacks. Rule *C-04* requires that initialization vectors (IVs) in CBC and CFB modes be random and unique. Complementarily, rule *C-05* enforces IV uniqueness across block cipher modes. In modes such as CTR and OFB, the IV must be unique for each encryption. In addition, rule *C-06* addresses asymmetric encryption padding, recommending modern schemes such as PSS and OAEP over the legacy PKCS1-v1.5 algorithm.

Key Lifecycle Security. *C-07* and *C-08* concern the use of predictable keys and salts in cryptographic functions, particularly in key derivation. Such practices can weaken the randomness of keys and have a negative impact on the security of the application. *C-09* and *C-10* further require sufficient iteration counts and adequate key lengths in key derivation functions, thereby enhancing resistance against brute-force and related attacks by increasing the computational cost for adversaries.

Random Number Security. *C-11* advises against using unsafe random number generators, particularly unsafe pseudo-random number generators in software. In JavaScript cryptographic libraries, there are two types of PRNGs available: (a) Statistical PRNGs, such as `Math.random()`, rely on deterministic algorithms and can be predicted through statisti-

cal analysis, making them unsuitable for security-sensitive applications. (b) Cryptographic PRNGs provided by cryptographic libraries generally offer stronger randomness and can meet security requirements. However, caution is still required, as some third-party implementations (e.g., `CryptoJS`) may fallback to weak randomness sources such as time- and counter-based generators, which introduce security risks.

Cryptographic Protocol Security. Secure configuration of cryptographic protocol suites is essential to protect communication channels. *C-12* prohibits the use of insecure HTTP parsers to prevent the acceptance of malicious input. *C-13* requires strict certificate validation to ensure that only legitimate entities are authenticated and allowed to establish connections. *C-14* discourages the use of weak SSL/TLS versions (e.g., SSLv3, TLS 1.0, TLS 1.1), which are vulnerable to attacks such as BEAST, BREACH, and CRIME [40].

Secure Storage of Sensitive Parameters. *C-15* and *C-16* emphasize the importance of implementing proper security measures when storing sensitive security parameters in cookies. In particular, precautions should be taken to mitigate XSS and related client-side attacks. Therefore, user identity tokens and session identifiers should be stored in cookies with the `HttpOnly` attribute enabled, ensuring that they cannot be accessed or modified by client-side scripts. Prior studies have demonstrated that disabling `HttpOnly` allows JavaScript to read or tamper with cookie attributes, which significantly enlarges the attack surface. Moreover, insecure cookie handling may facilitate CSRF exploitation, where a compromised JWT stored in a cookie can be leveraged to initiate forged requests on behalf of the victim.

A.2 LLM Prompts for Generation

LLM-assisted API Semantic Mapping. Figure 6 presents the prompt used to map natural-language cryptographic misuse constraints to the corresponding cryptographic libraries, and to identify and label APIs that are likely to serve as potential sources or sinks.

Rule Construction and Labeling. Figure 7 presents the prompt used to generate taint analysis rules for *JScamd* with four sections, the task instruction, knowledge context, framework constraint, and one-shot demonstration.

A.3 CVE List

During the early stage of this research, we developed a prototype of *JScamd* and conducted functional validation on several high-impact projects, which resulted in 10 assigned CVEs. After completing the comprehensive design and evaluation of *JScamd*, we submitted 31 additional CVE requests. Table 8 reports only a representative subset of the assigned CVEs, while the complete and continuously updated CVE list is provided in the open-source repository.

Imagine you are an expert in JavaScript security analysis and cryptographic misuse detection.
You are given: (1) a candidate API from {JavaScript cryptographic library}, and (2) a natural-language misuse constraint description.

Your task is to determine whether this API semantically matches the misuse constraint description. If a match exists, identify which API parameters participate in the misuse behavior and explain the semantic relevance. If no match exists, clearly state that the API is not relevant. You must reason based only on the provided information without inventing APIs, adding non-existent behaviors, or making unfounded assumptions.

Input API: {JScamd_extracted_API_Info}
Misuse Rule Description:{cryptographic_constraint}

[Output Format]
Semantic Match: YES / NO
If YES, provide structured mapping:
Matched API: {api_name}
Relevant Parameters: [{parameter_name_1}, {parameter_name_2}, ...]
Misuse Semantics: {{short justification of why this API matches}}
Security Impact: {{concise statement of the vulnerability behavior}}

If NO, output:
Semantic Match: NO
Reason: {{brief justification}}

Output strictly in the above format.

Figure 6: LLM prompt for LLM-based semantic mapping between cryptographic misuse constraints and library APIs.

Imagine you are skilled in JavaScript security analysis and cryptographic API misuse detection. Your task is to map cryptographic misuse patterns to the {JavaScript cryptographic library} API and generate taint-analysis detection rules.

Your generated rules must be semantically correct, aligned with {JavaScript cryptographic library} semantics. Each rule should correctly define source, sink, filtering logic, the judge function contains {Judge_function_in_JScamd} strictly following the required rule structure.

Rule structure definition:
{JScamd_rule_structure}
Example generated rule:
{example_rule}

Input security constraint misuse description:
{constraint_here}
Input {JavaScript cryptographic library} API information:
{API_Info}

Output strictly in the above format.

Figure 7: LLM prompt for taint analysis rule generation.

Table 8: Representative CVEs assigned through our disclosure process.

CVE	ID	Severity	Product (#Star)
CVE-2023-50475	C-01	critical	bcoin (2.9k)
CVE-2023-51837	C-13	critical	MeshCentral (2.5k)
CVE-2023-51839	C-03	critical	stf (2k)
CVE-2023-51840	C-07	critical	DoraCMS (3.4k)
CVE-2023-51838	C-01	high	MeshCentral (2.5k)
CVE-2023-51842	C-14	high	MeshCentral (2.5k)
CVE-2023-51843	C-15	high	dashboard (1.5k)